

Question no 1

A university wants to deploy a scalable virtualized environment for running 200+ student VMs simultaneously during a cloud computing lab. You are required to:

1. Compare Type-1 vs Type-2 hypervisors for this scenario in terms of:
 - o Hardware utilization
 - o Latency o Security
 - o VM isolation
2. Conclude with a justified recommendation for the university.

1. Comparison

a. Hardware Utilization

- **Type-1 Hypervisor (Bare-Metal):**
Runs directly on the server hardware without a host OS, so it uses CPU, memory, and I/O resources very efficiently. It can support a large number of VMs, which is ideal for running 200+ student VMs.
- **Type-2 Hypervisor (Hosted):**
Runs on top of a host operating system. Because of this extra layer, more resources are consumed by the OS itself, leading to lower VM density and inefficient resource utilization.

b. Latency

- **Type-1:**
Has very low latency since it communicates directly with hardware. This leads to faster VM performance and smooth operation in a lab environment.
- **Type-2:**
Higher latency because all VM operations go through the host OS first, which slows down response time.

c. Security

- **Type-1:**
More secure because it has a small attack surface and does not rely on a full operating system. Security vulnerabilities in host OS do not exist here.
- **Type-2:**
Less secure, as the host OS could be compromised. If the host OS becomes infected, all VMs on top of it are at risk.

d. VM Isolation

- **Type-1:**
Provides strong isolation between VMs. Problems in one VM rarely affect others. Ideal for multi-user environments like cloud labs.
- **Type-2:**
Isolation depends heavily on the host OS. Failure or crash in the host OS can shut down all VMs at once.

2. Conclusion and Recommendation

For a university that needs to run **200+ student virtual machines at the same time**, a **Type-1 hypervisor** is the best choice. It offers:

- Higher performance and better hardware utilization
- Lower latency
- Stronger security
- Excellent VM isolation

Type-1 hypervisors such as **VMware ESXi, Microsoft Hyper-V, Proxmox VE, or KVM** are specifically designed for large-scale, high-performance virtualized environments.

Final Recommendation:

The university should deploy a **Type-1 hypervisor–based virtualized cluster** to ensure scalability, reliability, and secure handling of 200+ student VMs.

Question no 2

Design a high-availability VM migration strategy for a data center using the following technologies:

- Live Migration (VMotion)
- Distributed Resource Scheduler (DRS)
- Shared Storage (SAN/NAS)

Explain:

How the system would automatically react when a physical host becomes overloaded.

1. Conditions under which DRS would not trigger migrations.

2. How memory pre-copying and dirty-page tracking influence migration time.
3. A failure case where VMotion may fail, and propose mitigation.

High-Availability VM Migration Strategy

A data center can achieve high availability by combining **Live Migration (VMotion)**, **DRS**, and **shared storage (SAN/NAS)**. All hosts access the same storage, so VMs can move between hosts without shutting down.

How the System Reacts When a Host Is Overloaded

- DRS continuously checks host CPU and RAM usage.
- When a host becomes overloaded, DRS selects VMs causing the load.
- VMotion then **live-migrates** these VMs to other hosts with free resources.
- The migration happens without downtime, restoring cluster balance automatically.

1. When DRS Does NOT Trigger Migration

- No other host has enough free CPU/RAM.
- Affinity/anti-affinity rules restrict VM movement.
- VM uses **local storage** or **GPU passthrough**.
- VM has a **local ISO mounted**.
- Hosts have **incompatible network or CPU settings**.
- DRS is in **manual mode**.

2. Effect of Memory Pre-Copy & Dirty Pages

- VMotion copies VM memory while the VM is still running.
- Pages modified during migration become **dirty pages** and must be recopied.
- High dirty-page rates (e.g., busy databases) increase migration time.
- If too many pages change, the migration may slow down or time out.

3. VMotion Failure Case & Mitigation

Failure Example:

Migration fails if the destination host lacks the same **network VLAN/port group**.

Mitigation:

- Keep all host network settings consistent using templates or automation.
 - Ensure every host supports the same VLANs, switches, and configurations.
-

Question no 3

Organizations often shift from VMs to containers for microservices. Provide a deep analytical comparison of:

- Namespace & cgroup isolation
- Container image layering
- VM vs container startup semantics
- Security boundary differences
- Microservice load balancing implications

Also discuss when containers perform worse than VMs, despite being "lightweight".

1. Namespace & cgroup isolation

Containers use Linux **namespaces** for process/network isolation and **cgroups** to limit CPU, RAM, and I/O. They share the host kernel, so isolation is lighter.

VMs run a **full guest OS** with hardware-level isolation, giving stronger separation.

2. Container image layering

Containers use a **layered filesystem** (base layer → app layers). This reduces storage, speeds up pulling, and makes images reusable.

VMs use large monolithic disk images that are slower to copy.

3. Startup semantics

Containers start in **milliseconds** because they only launch a process.

VMs take **seconds to minutes** because the entire OS boots.

This makes containers ideal for fast scaling in microservices.

4. Security boundary differences

Containers share a kernel, so a kernel exploit can break isolation. Security depends on policies (seccomp, AppArmor).

VMs provide stronger isolation because each VM has its own kernel protected by the hypervisor.

5. Microservice load balancing implications

Containers work well with microservice load balancing because they scale quickly and integrate with orchestration systems (Kubernetes).

VMs scale slowly, so load balancing is more static.

When containers perform worse than VMs

Containers can be slower when:

- Workloads need **heavy disk I/O** (databases).
 - Strong isolation or **multi-tenant security** is required.
 - Apps need **special kernel modules** or hardware.
 - Image pulls cause **cold start delays**.
-

Question no 4

Consider an MPI program with P processes, where each process must communicate its computed partial sum to every other process (all-to-all communication using point-to-point MPI_Send and MPI_Recv).

- Derive the total number of point-to-point communication operations.
- Discuss the scalability limitations of this approach for $P = 512$.
- Suggest a more efficient MPI approach (e.g., MPI_Allgather or MPI_Reduce_scatter) and justify the improvement using complexity terms.

1. Total Number of Point-to-Point Operations

For P processes:

- Each sends to $(P - 1)$ processes
- Each receives from $(P - 1)$ processes

Total operations:

$$\text{Operations} = P(P-1)$$

For both send and receive counted separately:

$$\text{Total Ops} = 2P(P-1)$$

2. Scalability Limitations for $P = 512$

- Complexity = $O(P^2)$
- 512 processes $\rightarrow 512 \times 511 \approx \mathbf{261,000 \text{ sends}}$
- Causes:
 - Network congestion

- High latency
- Large synchronization overhead
- Memory pressure

This becomes impractical for large clusters.

3. More Efficient Alternative

Use collective operations:

- **MPI_Allgather** → $O(\log P)$ or $O(P)$ depending on implementation
- **MPI_Reduce_scatter** → reduces data before exchanging

Why better?

Collectives use optimized tree or ring algorithms → fewer messages and better bandwidth utilization.

Question no 5

You are designing a cluster of 16 nodes executing an MPI-based weather simulation model.

Explain:

1. How domain decomposition would be used for distributing atmospheric grid data.
2. The implications of boundary data exchange (halo swapping).
3. How network topology (fat-tree vs torus) affects message latency.
4. Why collective communication may become a bottleneck when prediction frequency increases.
5. A theoretical performance model using $T = T_s + T_c(P)$, where:

T_s = sequential computation

T_c = communication overhead depending on P

You must express assumptions clearly and justify each term in the model.

1. Domain Decomposition

- Atmospheric 3D grid is divided into sub-domains.

- Each node gets a block of grid cells.
- Processes compute weather predictions only for their assigned block.

2. Boundary Data Exchange (Halo Swapping)

- Neighboring subdomains share border cells.
- These halo cells allow correct simulation near boundaries.
- Requires frequent communication → adds latency.

3. Effect of Network Topology

Fat-Tree:

- High bandwidth, low contention
- Good for all-to-all and heavy communication workloads

Torus:

- Only neighbor nodes directly connected
- Good for structured grid computations
- Higher hop count → higher latency for distant nodes

4. Why Collective Communication Becomes Bottleneck

When prediction frequency increases:

- Collectives (broadcast, reduce, gather) run more often
- Latency compounds
- Network saturation occurs
- Global synchronization slows simulation progress

5. Performance Model

$$T = T_s + T_c(P)$$

Assumptions:

- T_s = time spent in local computation per node
- $T_c(P)$ = communication time depending on number of processes P

$$T_c(P) = \alpha \log P + \beta \cdot \text{message size}$$

Where:

- α = latency per message
- β = time per byte transferred

Justification:

- Computation scales well with more nodes $\rightarrow T_s$ decreases.
- Communication grows with more processes due to halo exchange and collectives.
- Overall performance limited by $T_c(P)$ for large P .